

SUPPLY CHAIN CONTROL TOWER PROJECT

02_PowerBI_SQL_Validation

Phase 3 — Power BI KPI & Visualization SQL Validation (All 7 Dashboards)

SQLite Database | Star Schema Data Model

SUPPLY CHAIN CONTROL TOWER PROJECT

02_PowerBI_SQL_Validation.sql

Purpose:

This file contains the SQL validation for every Power BI KPI and every visualization used in the Supply Chain Control Tower, exactly as documented in the Supply Chain Control Tower Project Charter (Phase 3: Power BI Dashboard).

Database Engine : SQLite

Data Modeling : Star Schema

Fact Tables -> Orders, Forecast, Transportation

Dimension Tables -> Product, Supplier, Warehouse,

Supplier_Product_Map (Bridge Table)

Dashboard Order (as delivered in the project):

PAGE 1 - Executive Dashboard (Executive Overview)

PAGE 2 - Inventory Dashboard

PAGE 3 - Transportation Dashboard

PAGE 4 - Supplier Dashboard

PAGE 5 - Forecast Dashboard

PAGE 6 - Order Fulfillment Dashboard

PAGE 7 - Warehouse Dashboard

Deliverables Validated: 30+ DAX Measures | 40+ SQL Validation Queries

Validation Method:

Each SQL query was executed in SQLite to validate the numerical accuracy of the equivalent Power BI DAX measure or visualization. Any calculated columns or relationships used in Power BI were reproduced in SQL using joins, aggregations or expressions where applicable.

PAGE 1

Executive Dashboard

Dashboard Objective:

```
-- The Executive Overview dashboard provides senior management with a
-- consolidated view of overall supply chain performance. It combines
-- order fulfillment, inventory health, transportation, and supplier
-- metrics to monitor operational efficiency and identify high-priority
-- areas requiring business attention.
```

KPI 1

Total Orders**Power BI DAX:**

```
-- Total Orders =
-- DISTINCTCOUNT(Orders[Order_ID])
```

Purpose:

```
-- Measures the total number of unique customer orders processed.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Order_ID) AS Total_Orders
FROM Orders;
```

Expected Result:

```
-- Approximately 25K customer orders were processed across the supply
-- chain network.
```

KPI 2

Fill Rate %**Power BI DAX:**

```
-- Fill Rate % =
-- VAR Delivered = [Total Delivered]
-- VAR Demand = [Total Demand Qty]
-- RETURN
-- DIVIDE(Delivered, Demand, 0)
```

Purpose:

```
-- Measures the percentage of customer demand fulfilled.
```

Equivalent SQL:

```
SELECT
    ROUND(
        SUM(Delivered_Qty) * 100.0 /
        SUM(Demand_Qty), 2
    ) AS Fill_Rate
FROM Orders;
```

Expected Result:

```
-- Overall Fill Rate is 66.9%, indicating opportunities to improve order
-- fulfillment.
```

KPI 3***Delayed Orders*****Power BI DAX:**

```
-- Delayed Orders =
-- CALCULATE(
--     COUNTROWS(Orders),
--     REMOVEFILTERS(Orders[Order_Status]),
--     Orders[Order_Status]="DELAYED"
-- )
```

Purpose:

```
-- Counts the number of delayed customer orders.
```

Equivalent SQL:

```
SELECT
    COUNT(*) AS Delayed_Orders
FROM Orders
WHERE Order_Status = 'DELAYED';
```

Expected Result:

```
-- Approximately 8K customer orders experienced delivery delays.
```

KPI 4***At Risk SKUs*****Power BI DAX:**

```
-- At Risk SKUs =
-- CALCULATE(
--     DISTINCTCOUNT(Inventory[SKU_ID]),
--     Inventory[Inventory Status]="At Risk"
-- )
```

Purpose:

```
-- Counts the number of inventory items below safety stock.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT SKU_ID) AS At_Risk_SKUs
FROM Inventory
WHERE Current_Stock < Safety_Stock;
```

Expected Result:

```
-- 234 SKUs are currently below safety stock and require replenishment.
```

KPI 5**Total Freight Cost****Power BI DAX:**

```
-- Total Freight Cost =
-- SUM(Transportation[Freight_Cost])
```

Purpose:

```
-- Measures the organization's total transportation spend.
```

Equivalent SQL:

```
SELECT
    SUM(Freight_Cost) AS Total_Freight_Cost
FROM Transportation;
```

Expected Result:

```
-- Total transportation expenditure reached approximately ₹52 Million.
```

KPI 6**Total Suppliers****Power BI DAX:**

```
-- Total Suppliers =
-- DISTINCTCOUNT(Supplier[Supplier_ID])
```

Purpose:

```
-- Displays the active supplier base supporting the supply chain.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Supplier_ID) AS Total_Suppliers
FROM Supplier;
```

Expected Result:

```
-- The supply network is supported by 50 suppliers.
```

KPI 7**Average Delivery Days****Power BI DAX:**

```
-- Avg Delivery Days =
-- AVERAGE(Transportation[Delivery_Days])
```

Purpose:

```
-- Measures the average shipment delivery time.
```

Equivalent SQL:

```
SELECT
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM Transportation;
```

Expected Result:

```
-- Average transportation lead time is 4.89 days.
```

Visual 1**Order Status Distribution****Power BI DAX:**

```
-- Total Orders =
-- DISTINCTCOUNT(Orders[Order_ID])
```

Purpose:

```
-- Shows the proportion of Delivered, Partial, and Delayed orders,
-- helping management understand overall order fulfillment performance.
```

Equivalent SQL:

```
SELECT
```

```

    Order_Status,
    COUNT(DISTINCT Order_ID) AS Orders
FROM Orders
GROUP BY Order_Status;

```

Expected Result:

```

-- A distribution of orders across Delivered, Partial, and Delayed
-- statuses, highlighting the scale of the fulfillment gap driving the
-- 66.9% Fill Rate.

```

Visual 2***Inventory Risk by Warehouse*****Power BI DAX:**

```

-- At Risk SKUs =
-- CALCULATE(
--     DISTINCTCOUNT(Inventory[SKU_ID]),
--     Inventory[Inventory Status]="At Risk"
-- )

```

Purpose:

```

-- Shows warehouses with the highest number of inventory items below
-- safety stock, helping prioritize replenishment activities.

```

Equivalent SQL:

```

SELECT
    Warehouse_ID,
    COUNT(DISTINCT SKU_ID) AS At_Risk_SKUs
FROM Inventory
WHERE Current_Stock < Safety_Stock
GROUP BY Warehouse_ID
ORDER BY At_Risk_SKUs DESC;

```

Expected Result:

```

-- WH7 has the highest inventory risk and should be prioritized for
-- inventory replenishment.

```

Visual 3***Freight Cost by Carrier*****Power BI DAX:**

```

-- Total Freight Cost =
-- SUM(Transportation[Freight_Cost])

```

Purpose:

```

-- Compares logistics expenditure across transportation carriers to
-- identify carriers contributing the highest logistics cost.

```

Equivalent SQL:

```

SELECT
    Carrier,
    SUM(Freight_Cost) AS Freight_Cost
FROM Transportation
GROUP BY Carrier
ORDER BY Freight_Cost DESC;

```

Expected Result:

```

-- DHL accounts for the highest freight expenditure, presenting an
-- opportunity for carrier cost optimization.

```

PAGE 2

Inventory Dashboard

Dashboard Objective:

- The Inventory Dashboard provides a comprehensive view of inventory health across warehouses by monitoring stock availability, inventory investment, replenishment risk, and supplier lead times. It helps warehouse managers and planners identify stock shortages, optimize inventory investment, and prioritize replenishment activities.

KPI 1

Total Current Stock

Power BI DAX:

- Total Current Stock =
- SUM(Inventory[Current_Stock])

Purpose:

- Displays the total inventory currently available across all warehouses.

Equivalent SQL:

```
SELECT
    SUM(Current_Stock) AS Total_Current_Stock
FROM Inventory;
```

Expected Result:

- Total current stock across all warehouses is approximately 2M units.

KPI 2

Total Safety Stock

Power BI DAX:

- Total Safety Stock =
- SUM(Inventory[Safety_Stock])

Purpose:

- Displays the total minimum stock maintained across the network.

Equivalent SQL:

```
SELECT
    SUM(Safety_Stock) AS Total_Safety_Stock
FROM Inventory;
```

Expected Result:

- Safety stock maintained across the network totals 523K units.

KPI 3

Total Inventory Value

Power BI DAX:

- Total Inventory Value =
- SUM(Inventory[Inventory_Value])

Purpose:

- Displays the monetary value of inventory currently held.

Equivalent SQL:

```
SELECT
    SUM(Inventory_Value) AS Total_Inventory_Value
FROM Inventory;
```

Expected Result:

```
-- Total inventory investment is approximately ₹463M.
```

KPI 4

At Risk SKUs

Power BI DAX:

```
-- At Risk SKUs =
-- CALCULATE(
--     DISTINCTCOUNT(Inventory[SKU_ID]),
--     Inventory[Inventory Status] = "At Risk"
-- )
```

Purpose:

```
-- Counts the distinct SKUs currently below safety stock.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT SKU_ID) AS At_Risk_SKUs
FROM Inventory
WHERE Current_Stock < Safety_Stock;
```

Expected Result:

```
-- 234 SKUs are currently below safety stock and require replenishment.
```

KPI 5

Healthy SKUs

Power BI DAX:

```
-- Healthy SKUs =
-- CALCULATE(
--     DISTINCTCOUNT(Inventory[SKU_ID]),
--     Inventory[Inventory Status] = "Healthy"
-- )
```

Purpose:

```
-- Counts inventory records meeting the safety stock requirement.
```

Equivalent SQL:

```
SELECT
    COUNT(*) AS Healthy_SKUs
FROM Inventory
WHERE Current_Stock >= Safety_Stock;
```

Expected Result:

```
-- Approximately 2.68K SKUs (89.17%) are operating within healthy
-- inventory levels.
```

KPI 6

Average Lead Time

Power BI DAX:

```
-- Avg Lead Time =
-- AVERAGE(Supplier[Lead_Time_Days])
```

Purpose:

```
-- Measures the average supplier replenishment lead time.
```

Equivalent SQL:

```
SELECT
    ROUND(AVG(Lead_Time_Days), 2) AS Avg_Lead_Time
FROM Supplier;
```

Expected Result:

```
-- The average supplier lead time across all SKUs is 14.66 days,
-- supporting replenishment planning.
```

Visual 1

Inventory Status Distribution

Power BI DAX:

```
-- Inventory Status =
-- IF(
--     Inventory[Current_Stock] < Inventory[Safety_Stock],
--     "At Risk",
--     "Healthy"
-- )
```

Purpose:

```
-- Displays the proportion of Healthy and At Risk inventory across all
-- warehouses, providing an overall inventory health indicator.
```

Equivalent SQL:

```
SELECT
    CASE
        WHEN Current_Stock < Safety_Stock THEN 'At Risk'
        ELSE 'Healthy'
    END AS Inventory_Status,
    COUNT(*) AS SKU_Count
FROM Inventory
GROUP BY Inventory_Status;
```

Expected Result:

```
-- Inventory health across warehouses remains strong, with nearly 90% of
-- inventory above safety stock.
```

Visual 2

Stock Availability by Warehouse

Power BI DAX:

```
-- Total Current Stock =
-- SUM(Inventory[Current_Stock])
-- Total Safety Stock =
-- SUM(Inventory[Safety_Stock])
```

Purpose:

```
-- Compares Current Stock with Safety Stock for each warehouse,
-- highlighting whether warehouses maintain sufficient stock above
-- minimum inventory requirements.
```

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    SUM(Current_Stock) AS Current_Stock,
    SUM(Safety_Stock) AS Safety_Stock
FROM Inventory
GROUP BY Warehouse_ID
ORDER BY Current_Stock DESC;
```

Expected Result:

```
-- Current stock significantly exceeds safety stock across warehouses,
-- indicating stable inventory availability.
```

Visual 3

Inventory Value by Warehouse

Power BI DAX:

```
-- Total Inventory Value =
-- SUM(Inventory[Inventory_Value])
```

Purpose:

- Shows the total inventory investment by warehouse, identifying
- warehouses with the highest capital tied up in inventory.

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    SUM(Inventory_Value) AS Inventory_Value
FROM Inventory
GROUP BY Warehouse_ID
ORDER BY Inventory_Value DESC;
```

Expected Result:

- WH3 and WH5 hold the highest inventory investment (approximately ₹61M
- each).

PAGE 3

Transportation Dashboard

Dashboard Objective:

- Monitor transportation performance by analyzing shipment volume,
- freight expenditure, carrier utilization, delivery performance, and
- transportation lanes to support logistics cost optimization and
- carrier performance management.

KPI 1

Total Shipments

Power BI DAX:

- Total Shipments =
- COUNTROWS(Transportation)

Purpose:

- Displays the total number of shipments handled.

Equivalent SQL:

```
SELECT
    COUNT(*) AS Total_Shipments
FROM Transportation;
```

Expected Result:

- Approximately 5,000 shipments were executed across the logistics
- network.

KPI 2

Total Freight Cost

Power BI DAX:

- Total Freight Cost =
- SUM(Transportation[Freight_Cost])

Purpose:

- Displays total transportation expenditure.

Equivalent SQL:

```
SELECT
    SUM(Freight_Cost) AS Total_Freight_Cost
FROM Transportation;
```

Expected Result:

- Total freight expenditure reached ₹52M.

KPI 3

Average Freight Cost

Power BI DAX:

- Avg Freight Cost =
- AVERAGE(Transportation[Freight_Cost])

Purpose:

- Measures the average transportation cost per shipment.

Equivalent SQL:

```
SELECT
    AVG(Freight_Cost) AS Avg_Freight_Cost
FROM Transportation;
```

Expected Result:

- Average freight cost per shipment was ₹10.41K.

KPI 4***Average Delivery Days*****Power BI DAX:**

```
-- Avg Delivery Days =
-- AVERAGE(Transportation[Delivery_Days])
```

Purpose:

```
-- Measures average transportation lead time.
```

Equivalent SQL:

```
SELECT
    AVG(Delivery_Days) AS Avg_Delivery_Days
FROM Transportation;
```

Expected Result:

```
-- Average delivery performance remained stable at 4.89 days.
```

KPI 5***Total Carriers*****Power BI DAX:**

```
-- Total Carriers =
-- DISTINCTCOUNT(Transportation[Carrier])
```

Purpose:

```
-- Displays the number of logistics partners.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Carrier) AS Total_Carriers
FROM Transportation;
```

Expected Result:

```
-- Transportation operations utilized 4 logistics carriers.
```

KPI 6***Total Routes*****Power BI DAX:**

```
-- Total Routes =
-- DISTINCTCOUNT(Transportation[Origin] & " → " & Transportation[Destination])
```

Purpose:

```
-- Shows the number of transportation lanes available.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Origin || ' -> ' || Destination) AS Total_Routes
FROM Transportation;
```

Expected Result:

```
-- Transportation operations utilized 3 transportation routes.
```

Visual 1***Freight Spend by Carrier*****Power BI DAX:**

```
-- Total Freight Cost =
-- SUM(Transportation[Freight_Cost])
```

Purpose:

```
-- Compares transportation spending across logistics partners to
-- identify the most expensive carrier and evaluate freight cost
-- distribution.
```

Equivalent SQL:

```
SELECT
    Carrier,
    SUM(Freight_Cost) AS Total_Freight_Cost
FROM Transportation
GROUP BY Carrier
ORDER BY Total_Freight_Cost DESC;
```

Expected Result:

```
-- DHL recorded the highest freight expenditure (~₹13.2M). Freight costs
-- are relatively balanced across carriers, indicating diversified
-- transportation spending.
```

Visual 2***Shipment Count by Carrier*****Power BI DAX:**

```
-- Shipment Count =
-- COUNTROWS(Transportation)
-- (or)
-- Shipment Count =
-- DISTINCTCOUNT(Transportation[Shipment_ID])
```

Purpose:

```
-- Compares shipment volume handled by each carrier to evaluate carrier
-- utilization and identify dependency on specific logistics partners.
```

Equivalent SQL:

```
SELECT
    Carrier,
    COUNT(*) AS Shipment_Count
FROM Transportation
GROUP BY Carrier
ORDER BY Shipment_Count DESC;
```

Expected Result:

```
-- Shipment volume distribution highlights carrier utilization across
-- the network; carrier workload appears relatively balanced, reducing
-- dependency on a single logistics provider.
```

Visual 3***Carrier Performance by Lane (Matrix)*****Power BI DAX:**

```
-- Avg Delivery Days =
-- AVERAGE(Transportation[Delivery_Days])
```

Purpose:

```
-- Evaluates carrier performance across transportation lanes by
-- comparing average delivery time by Origin (or Route) and Carrier to
-- identify the best-performing carrier for each lane.
```

Equivalent SQL:

```
SELECT
    Origin,
    Carrier,
    ROUND(AVG(Delivery_Days), 2) AS Avg_Delivery_Days
FROM Transportation
GROUP BY Origin, Carrier
ORDER BY Origin, Carrier;
```

Expected Result:

```
-- Delivery performance differs across transportation lanes; certain
-- carrier-route combinations consistently achieve lower delivery times,
-- supporting lane-specific carrier assignment to improve service levels
```

-- and transportation efficiency.

PAGE 4

Supplier Dashboard

Dashboard Objective:

- Evaluate supplier performance by analyzing supplier reliability,
- customer demand fulfillment, on-time delivery (OTD), lead time,
- defect rate, and supplier risk to support supplier selection,
- performance improvement, and procurement decisions.

KPI 1

Total Suppliers

Power BI DAX:

- Total Suppliers =
- DISTINCTCOUNT(Supplier[Supplier_ID])

Purpose:

- Displays the total number of active suppliers.

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Supplier_ID) AS Total_Suppliers
FROM Supplier;
```

Expected Result:

- The supply base consists of 50 suppliers.

KPI 2

Total Demand Qty

Power BI DAX:

- Total Demand Qty =
- SUM(Orders[Demand_Qty])

Purpose:

- Displays the total customer demand supported by suppliers.

Equivalent SQL:

```
SELECT
    SUM(Demand_Qty) AS Total_Demand_Qty
FROM Orders;
```

Expected Result:

- Suppliers support approximately 4M units of customer demand.

KPI 3

Average OTD %

Power BI DAX:

- Average OTD % =
- DIVIDE(
- AVERAGE(Supplier[OTD_Pct]),
- 100
-)

Purpose:

- Measures supplier delivery reliability.

Equivalent SQL:

```
SELECT
    AVG(OTD_Pct) AS Avg_OTD_Percentage
FROM Supplier;
```

Expected Result:

```
-- Suppliers achieved an average OTD of 77.98%, indicating opportunities
-- to improve delivery reliability.
```

KPI 4***Average Lead Time*****Power BI DAX:**

```
-- Avg Lead Time =
-- AVERAGE(Supplier[Lead_Time_Days])
```

Purpose:

```
-- Measures the average supplier replenishment lead time.
```

Equivalent SQL:

```
SELECT
    AVG(Lead_Time_Days) AS Avg_Lead_Time
FROM Supplier;
```

Expected Result:

```
-- Average supplier lead time is 14.7 days.
```

KPI 5***Average Defect Rate*****Power BI DAX:**

```
-- Average Defect Rate =
-- DIVIDE(
--     AVERAGE(Supplier[Defect_Rate]),
--     100
-- )
```

Purpose:

```
-- Measures average supplier quality performance.
```

Equivalent SQL:

```
SELECT
    AVG(Defect_Rate) AS Avg_Defect_Rate
FROM Supplier;
```

Expected Result:

```
-- Average supplier defect rate stands at 4.61%.
```

KPI 6***High Risk Suppliers*****Power BI DAX:**

```
-- High Risk Suppliers =
-- CALCULATE(
--     DISTINCTCOUNT(Supplier[Supplier_ID]),
--     FILTER(
--         Supplier,
--         Supplier[OTD_Pct] < 75 ||
--         Supplier[Lead_Time_Days] > 20 ||
--         Supplier[Defect_Rate] > 5
--     )
-- )
```

Purpose:

```
-- Identifies suppliers requiring immediate performance review based on
-- service and quality thresholds.
```

Risk Criteria: OTD < 75%, Lead Time > 20 Days, Defect Rate > 5%.

Equivalent SQL:

```

SELECT
    COUNT(DISTINCT Supplier_ID) AS High_Risk_Suppliers
FROM Supplier
WHERE
    OTD_Pct < 75
    OR Lead_Time_Days > 20
    OR Defect_Rate > 5;

```

Expected Result:

```

-- 38 suppliers meet at least one high-risk criterion, indicating
-- significant supplier performance improvement opportunities.

```

Visual 1***Top 10 Suppliers by Customer Demand*****Power BI DAX:**

```

-- Total Demand Qty =
-- SUM(Orders[Demand_Qty])

```

Purpose:

```

-- Identifies suppliers supporting the highest customer demand,
-- highlighting strategic suppliers and potential dependency risks.

```

Equivalent SQL:

```

SELECT
    s.Supplier_Name,
    SUM(o.Demand_Qty) AS Total_Demand
FROM Orders o
JOIN Supplier_Product_Map sp
    ON o.SKU_ID = sp.SKU_ID
JOIN Supplier s
    ON sp.Supplier_ID = s.Supplier_ID
GROUP BY s.Supplier_Name
ORDER BY Total_Demand DESC
LIMIT 10;

```

Expected Result:

```

-- Supplier_38 supports the highest customer demand (~144K units);
-- customer demand is concentrated among a limited number of suppliers,
-- indicating potential dependency risk.

```

Visual 2***Top 10 Suppliers by OTD %*****Power BI DAX:**

```

-- Average OTD % =
-- DIVIDE(
--     AVERAGE(Supplier[OTD_Pct]),
--     100
-- )

```

Purpose:

```

-- Identifies the most reliable suppliers based on On-Time Delivery
-- performance.

```

Equivalent SQL:

```

SELECT
    Supplier_Name,
    OTD_Pct
FROM Supplier
ORDER BY OTD_Pct DESC
LIMIT 10;

```

Expected Result:

```

-- The highest-performing suppliers consistently achieve 90-97% On-Time

```



```
-- Delivery; these suppliers represent benchmarks for delivery
-- performance.
```

Visual 3

Supplier Risk Matrix

Power BI DAX:

```
-- None (Scatter chart uses raw columns.)
```

Purpose:

```
-- Evaluates supplier risk using Lead Time and Defect Rate to identify
-- suppliers with both long lead times and high defect rates for
-- proactive supplier development and risk mitigation.
```

Equivalent SQL:

```
SELECT
    Supplier_Name,
    Lead_Time_Days,
    Defect_Rate
FROM Supplier;
```

Expected Result:

```
-- Suppliers located in the upper-right quadrant (high lead time and
-- high defect rate) represent the greatest operational risk; suppliers
-- with low lead time and low defect rates demonstrate stable and
-- reliable performance.
```

PAGE 5

Forecast Dashboard

Dashboard Objective:

- Evaluate forecast planning performance by comparing forecasted demand
- with actual demand, measuring forecast accuracy, identifying forecast
- variance across warehouses and months, and supporting demand planning
- improvements.

KPI 1

Total Forecast Qty

Power BI DAX:

- Total Forecast Qty =
- SUM(Forecast[Forecast_Qty])

Purpose:

- Displays the total forecasted demand.

Equivalent SQL:

```
SELECT
    SUM(Forecast_Qty) AS Total_Forecast_Qty
FROM Forecast;
```

Expected Result:

- Total forecast volume closely matched actual demand at approximately
- 1M units.

KPI 2

Total Actual Demand

Power BI DAX:

- Total Actual Demand =
- SUM(Forecast[Actual_Demand])

Purpose:

- Displays total customer demand recorded.

Equivalent SQL:

```
SELECT
    SUM(Actual_Demand) AS Total_Actual_Demand
FROM Forecast;
```

Expected Result:

- Actual demand closely aligns with forecasted demand at approximately
- 1M units.

KPI 3

Forecast Variance

Power BI DAX:

- Forecast Variance =
- [Total Forecast Qty] -
- [Total Actual Demand]
- (Positive Value -> Over Forecast; Negative Value -> Under Forecast)

Purpose:

- Measures deviation between forecast and actual demand.

Equivalent SQL:

```
SELECT
    SUM(Forecast_Qty) -
```

```
SUM(Actual_Demand) AS Forecast_Variance
FROM Forecast;
```

Expected Result:

```
-- The resulting forecast variance is only -1K units, indicating a
-- slight underforecast at the network level.
```

KPI 4**Forecast Accuracy %****Power BI DAX:**

```
-- Forecast Accuracy % =
-- VAR ForecastError =
--     DIVIDE(
--         ABS([Forecast Variance]),
--         [Total Actual Demand]
--     )
-- VAR Accuracy =
--     1 - ForecastError
-- RETURN
--     MAX(0, Accuracy)
```

Purpose:

```
-- Measures how closely forecast matches actual demand. A higher
-- percentage indicates better demand planning performance.
```

Equivalent SQL:

```
SELECT
    ROUND(
        (
            1 -
            (
                ABS(SUM(Forecast_Qty) - SUM(Actual_Demand))
                * 100.0
                / SUM(Actual_Demand)
            ) / 100
        ) * 100,
        2
    ) AS Forecast_Accuracy_Pct
FROM Forecast;
```

Expected Result:

```
-- Overall forecast accuracy reached 99.92%, indicating highly reliable
-- demand planning.
```

KPI 5**Forecast Error %****Power BI DAX:**

```
-- Forecast Error % =
-- DIVIDE(
--     ABS([Forecast Variance]),
--     [Total Actual Demand]
-- )
```

Purpose:

```
-- Shows the forecasting deviation percentage. Lower values indicate
-- better planning accuracy.
```

Equivalent SQL:

```
SELECT
    ABS(
        SUM(Forecast_Qty) - SUM(Actual_Demand)
    )
```

```

    )
    / SUM(Actual_Demand)
    AS Forecast_Error
FROM Forecast;

```

Expected Result:

```
-- Forecast error remained exceptionally low at 0.08%.
```

KPI 6**Total Warehouses****Power BI DAX:**

```
-- Total Warehouses =
-- DISTINCTCOUNT(Inventory[Warehouse_ID])
```

Purpose:

```
-- Displays the number of warehouses included in forecasting.
```

Equivalent SQL:

```

SELECT
    COUNT(DISTINCT Warehouse_ID) AS Total_Warehouses
FROM Inventory;

```

Expected Result:

```
-- Forecast planning covered 8 warehouses across the distribution
-- network.
```

Visual 1**Forecast vs Actual Demand****Power BI DAX:**

```
-- Total Forecast Qty =
-- SUM(Forecast[Forecast_Qty])
-- Total Actual Demand =
-- SUM(Forecast[Actual_Demand])
```

Purpose:

```
-- Compares planned demand against actual demand across warehouses,
-- highlighting forecast deviation at warehouse level and identifying
-- locations requiring planning improvements.
```

Equivalent SQL:

```

SELECT
    Warehouse_ID,
    SUM(Forecast_Qty) AS Forecast_Qty,
    SUM(Actual_Demand) AS Actual_Demand
FROM Forecast
GROUP BY Warehouse_ID
ORDER BY Actual_Demand DESC;

```

Expected Result:

```
-- Forecasted demand closely aligned with actual demand across all
-- warehouses; minor deviations suggest effective planning and stable
-- customer demand, with no warehouse experiencing significant
-- over-forecasting or under-forecasting.
```

Visual 2**Forecast Accuracy by Warehouse****Power BI DAX:**

```
-- Forecast Accuracy %
-- (see KPI 4 definition above)
```

Purpose:

```
-- Compares planning accuracy across warehouses, identifying warehouses
```

-- with the most reliable demand forecasting.

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    ROUND(
        CASE
            WHEN (
                1 -
                (
                    ABS(SUM(Forecast_Qty) - SUM(Actual_Demand))
                    * 1.0
                    / SUM(Actual_Demand)
                )
            ) < 0
            THEN 0
            ELSE (
                1 -
                (
                    ABS(SUM(Forecast_Qty) - SUM(Actual_Demand))
                    * 1.0
                    / SUM(Actual_Demand)
                )
            )
        END * 100,
        2
    ) AS Forecast_Accuracy_Pct
FROM Forecast
GROUP BY Warehouse_ID
ORDER BY Forecast_Accuracy_Pct DESC;
```

Expected Result:

-- WH8 achieved the highest forecast accuracy (99.92%), demonstrating
 -- excellent planning performance; all warehouses maintained forecast
 -- accuracy above 95%, indicating consistent planning quality across the
 -- network.

Visual 3

Monthly Forecast Variance

Power BI DAX:

-- Forecast Variance
 -- (see KPI 3 definition above)

Purpose:

-- Tracks forecasting deviation across months, identifying months where
 -- forecasting significantly exceeded or fell below actual demand.

Equivalent SQL:

```
SELECT
    Month,
    SUM(Forecast_Qty) - SUM(Actual_Demand)
    AS Forecast_Variance
FROM Forecast
GROUP BY Month
ORDER BY
    CASE Month
        WHEN 'Jan' THEN 1
        WHEN 'Feb' THEN 2
        WHEN 'Mar' THEN 3
        WHEN 'Apr' THEN 4
        WHEN 'May' THEN 5
        WHEN 'Jun' THEN 6
```

END;

Expected Result:

- April recorded the highest positive forecast variance (+9.6K),
- indicating over-forecasting; May experienced the largest negative
- variance (-8.4K), indicating under-forecasting. Monthly variance
- remained relatively small overall, reflecting stable forecasting
- performance.

PAGE 6

Order Fulfillment Dashboard

Dashboard Objective:

- Monitor order fulfillment performance by evaluating customer demand,
- delivered quantity, fill rate, delayed orders, regional performance,
- warehouse service levels, and product-level fulfillment to improve
- customer service and inventory availability.

KPI 1

Total Orders

Power BI DAX:

- Total Orders =
- DISTINCTCOUNT(Orders[Order_ID])

Purpose:

- Displays the total number of customer orders processed.

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Order_ID) AS Total_Orders
FROM Orders;
```

Expected Result:

- The business processed approximately 25K customer orders.

KPI 2

Total Demand Qty

Power BI DAX:

- Total Demand Qty =
- SUM(Orders[Demand_Qty])

Purpose:

- Displays total customer demand.

Equivalent SQL:

```
SELECT
    SUM(Demand_Qty) AS Total_Demand_Qty
FROM Orders;
```

Expected Result:

- Total customer demand reached approximately 4M units.

KPI 3

Total Delivered

Power BI DAX:

- Total Delivered =
- SUM(Orders[Delivered_Qty])

Purpose:

- Displays total quantity successfully delivered.

Equivalent SQL:

```
SELECT
    SUM(Delivered_Qty) AS Total_Delivered
FROM Orders;
```

Expected Result:

- Around 3M units were successfully delivered.

KPI 4**Fill Rate %****Power BI DAX:**

```
-- Fill Rate % =
-- VAR Delivered = [Total Delivered]
-- VAR Demand = [Total Demand Qty]
-- RETURN
-- DIVIDE(Delivered, Demand, 0)
```

Purpose:

```
-- Measures the percentage of customer demand successfully fulfilled.
-- Higher values indicate better order fulfillment performance.
```

Equivalent SQL:

```
SELECT
    ROUND(
        SUM(Delivered_Qty) * 100.0 / SUM(Demand_Qty),
        2
    ) AS Fill_Rate_Pct
FROM Orders;
```

Expected Result:

```
-- Overall Fill Rate is 66.9%.
```

KPI 5**Delayed Order %****Power BI DAX:**

```
-- Delayed Order % =
-- DIVIDE(
--     [Delayed Orders],
--     CALCULATE(
--         [Total Orders],
--         REMOVEFILTERS(Orders[Order_Status])
--     ),
--     0
-- )
```

Purpose:

```
-- Measures the percentage of delayed customer orders. Lower values
-- indicate better service performance.
```

Equivalent SQL:

```
SELECT
    COUNT(CASE WHEN Order_Status = 'DELAYED' THEN 1 END) * 100.0 /
    COUNT(*) AS Delayed_Order_Percentage
FROM Orders;
```

Expected Result:

```
-- Approximately 33.12% of orders experienced delivery delays.
```

KPI 6**Regions Served****Power BI DAX:**

```
-- Regions Served =
-- DISTINCTCOUNT(Orders[Customer_Region])
```

Purpose:

```
-- Displays the number of customer regions served.
```

Equivalent SQL:

```
SELECT
```



```
COUNT(DISTINCT Customer_Region) AS Regions_Served
FROM Orders;
```

Expected Result:

```
-- Operations supported customers across 4 geographic regions.
```

Visual 1***Regional Demand vs Delivered Quantity*****Power BI DAX:**

```
-- Total Demand Qty =
-- SUM(Orders[Demand_Qty])
-- Total Delivered =
-- SUM(Orders[Delivered_Qty])
```

Purpose:

```
-- Compares customer demand and delivered quantity across regions,
-- highlighting fulfillment gaps between customer demand and delivered
-- quantity.
```

Equivalent SQL:

```
SELECT
    Customer_Region,
    SUM(Demand_Qty) AS Total_Demand,
    SUM(Delivered_Qty) AS Total_Delivered
FROM Orders
GROUP BY Customer_Region
ORDER BY Total_Demand DESC;
```

Expected Result:

```
-- All regions exhibit a noticeable gap between demand and delivered
-- quantity; North recorded the highest demand volume, followed closely
-- by West and East, suggesting network-wide fulfillment constraints
-- rather than a region-specific issue.
```

Visual 2***Fill Rate % by Product Category*****Power BI DAX:**

```
-- Fill Rate %
-- (see KPI 4 definition above)
```

Purpose:

```
-- Compares fulfillment performance across product categories,
-- identifying categories with lower service levels that require
-- inventory or replenishment improvements.
```

Equivalent SQL:

```
SELECT
    P.Category,
    ROUND(
        SUM(O.Delivered_Qty) * 100.0 /
        SUM(O.Demand_Qty),
        2
    ) AS Fill_Rate
FROM Orders O
JOIN Product P
    ON O.SKU_ID = P.SKU_ID
GROUP BY P.Category
ORDER BY Fill_Rate DESC;
```

Expected Result:

```
-- Fill rate performance remains relatively consistent across product
-- categories (66-68%); Furniture achieved the highest fill rate
```

- (~67.5%), while Appliances recorded the lowest, indicating balanced
- inventory allocation with no extreme category underperformance.

Visual 3

Warehouse Fill Rate %

Power BI DAX:

- Fill Rate %
- (see KPI 4 definition above)

Purpose:

- Compares warehouse fulfillment performance, identifying warehouses
- delivering the highest customer service levels.

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    ROUND(
        SUM(Delivered_Qty) * 100.0 /
        SUM(Demand_Qty),
        2
    ) AS Fill_Rate_Pct
FROM Orders
GROUP BY Warehouse_ID
ORDER BY Fill_Rate_Pct DESC;
```

Expected Result:

- Warehouse service levels range between 66.2% and 68.0%; WH5 achieved
- the highest fulfillment performance, while WH2 recorded the lowest.

Visual 4

Lowest Fill Rate Products

Power BI DAX:

- Fill Rate %
- (see KPI 4 definition above)

Purpose:

- Identifies products with the poorest fulfillment performance,
- highlighting products requiring inventory optimization, supplier
- review, or replenishment planning.

Equivalent SQL:

```
SELECT
    P.Product_Name,
    ROUND(
        SUM(O.Delivered_Qty) * 100.0 /
        SUM(O.Demand_Qty),
        2
    ) AS Fill_Rate_Pct
FROM Orders O
JOIN Product P
    ON O.SKU_ID = P.SKU_ID
GROUP BY P.Product_Name
ORDER BY Fill_Rate_Pct ASC
LIMIT 10;
```

Expected Result:

- Several products exhibit fill rates below 55%, indicating significant
- fulfillment challenges; these products should be prioritized for
- inventory replenishment, supplier review, and demand planning
- adjustments.

PAGE 7

Warehouse Dashboard

Dashboard Objective:

- Monitor warehouse inventory performance by evaluating stock
- availability, inventory value, warehouse utilization, inventory
- health, and high-value SKUs to support inventory optimization and
- replenishment planning.

KPI 1

Total Warehouses

Power BI DAX:

- Total Warehouses =
- DISTINCTCOUNT(Inventory[Warehouse_ID])

Purpose:

- Displays the number of warehouses in the supply chain network.

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT Warehouse_ID) AS Total_Warehouses
FROM Inventory;
```

Expected Result:

- The organization manages inventory across 8 warehouses.

KPI 2

Total Inventory

Power BI DAX:

- Total Inventory =
- SUM(Inventory[Current_Stock])

Purpose:

- Displays the total current inventory available across all
- warehouses.

Equivalent SQL:

```
SELECT
    SUM(Current_Stock) AS Total_Inventory
FROM Inventory;
```

Expected Result:

- Inventory holds approximately 2M units of stock.

KPI 3

Avg Inventory / Warehouse

Power BI DAX:

- Avg Inventory / WH =
- DIVIDE(
- [Total Inventory],
- [Total Warehouses],
- 0
-)

Purpose:

- Measures the average inventory held per warehouse.

Equivalent SQL:

```
SELECT
```

```
SUM(Current_Stock) * 1.0 /
COUNT(DISTINCT Warehouse_ID) AS Avg_Inventory_Per_Warehouse
FROM Inventory;
```

Expected Result:

```
-- The average inventory per warehouse is 286K units, indicating a
-- relatively balanced inventory distribution.
```

KPI 4***Avg Reorder Point*****Power BI DAX:**

```
-- Avg Reorder Point =
-- AVERAGE(Inventory[Reorder_Point])
```

Purpose:

```
-- Displays the average inventory level at which replenishment should
-- begin.
```

Equivalent SQL:

```
SELECT
    AVG(Reorder_Point) AS Avg_Reorder_Point
FROM Inventory;
```

Expected Result:

```
-- The average reorder point is 301 units, serving as the replenishment
-- trigger across SKUs.
```

KPI 5***At Risk SKUs*****Power BI DAX:**

```
-- At Risk SKUs =
-- CALCULATE(
--     DISTINCTCOUNT(Inventory[SKU_ID]),
--     Inventory[Inventory Status] = "At Risk"
-- )
```

Purpose:

```
-- Shows the number of unique SKUs currently below safety stock levels.
```

Equivalent SQL:

```
SELECT
    COUNT(DISTINCT SKU_ID) AS At_Risk_SKUs
FROM Inventory
WHERE Current_Stock < Safety_Stock;
```

Expected Result:

```
-- Approximately 234 SKUs are currently below safety stock and require
-- replenishment.
```

KPI 6***Total Inventory Value*****Power BI DAX:**

```
-- Total Inventory Value =
-- SUM(Inventory[Inventory_Value])
```

Purpose:

```
-- Displays the monetary value of inventory currently held.
```

Equivalent SQL:

```
SELECT
    SUM(Inventory_Value) AS Total_Inventory_Value
FROM Inventory;
```

Expected Result:

- Total inventory value is approximately ₹463M, representing
- significant working capital invested in inventory.

Visual 1***Inventory by Warehouse*****Power BI DAX:**

- Total Inventory
- (see KPI 2 definition above)

Purpose:

- Compares total inventory across warehouses to identify which
- warehouses hold the largest inventory volumes.

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    SUM(Current_Stock) AS Total_Inventory
FROM Inventory
GROUP BY Warehouse_ID
ORDER BY Total_Inventory DESC;
```

Expected Result:

- WH8 maintains the highest inventory level (301K units), while WH6
- holds the lowest (255K units); inventory is fairly balanced across
- warehouses, suggesting effective stock distribution.

Visual 2***Inventory Health by Warehouse*****Power BI DAX:**

- At Risk SKUs
- (see KPI 5 definition above)
- Healthy SKUs =
- CALCULATE(
- DISTINCTCOUNT(Inventory[SKU_ID]),
- Inventory[Inventory Status] = "Healthy"
-)

Purpose:

- Compares healthy and at-risk SKUs across warehouses, highlighting
- warehouses requiring replenishment attention and those maintaining
- healthy stock levels.

Equivalent SQL:

```
SELECT
    Warehouse_ID,
    COUNT(DISTINCT CASE
        WHEN Current_Stock < Safety_Stock
        THEN SKU_ID
    END) AS At_Risk_SKUs,
    COUNT(DISTINCT CASE
        WHEN Current_Stock >= Safety_Stock
        THEN SKU_ID
    END) AS Healthy_SKUs
FROM Inventory
GROUP BY Warehouse_ID
ORDER BY Warehouse_ID;
```

Expected Result:

- WH7 contains the highest number of at-risk SKUs (51), indicating the
- greatest replenishment priority; WH2 has the highest number of

```
-- healthy SKUs (261), reflecting strong inventory availability. Most
-- warehouses maintain substantially more healthy SKUs than at-risk
-- SKUs, indicating overall stable inventory health.
```

Visual 3

Inventory Value by Category

Power BI DAX:

```
-- Total Inventory Value
-- (see KPI 6 definition above)
```

Purpose:

```
-- Compares inventory investment across product categories to identify
-- which categories consume the largest portion of inventory capital.
```

Equivalent SQL:

```
SELECT
    P.Category,
    SUM(I.Inventory_Value) AS Inventory_Value
FROM Inventory I
JOIN Product P
    ON I.SKU_ID = P.SKU_ID
GROUP BY P.Category
ORDER BY Inventory_Value DESC;
```

Expected Result:

```
-- Parts account for the highest inventory investment (₹142M), followed
-- by Furniture (₹115M); Appliances contribute the lowest inventory
-- value (₹100M). Inventory capital is concentrated in a few key product
-- categories, highlighting opportunities for inventory optimization.
```

Visual 4

Top 10 Inventory Value SKUs

Power BI DAX:

```
-- Total Inventory Value
-- (see KPI 6 definition above)
```

Purpose:

```
-- Identifies SKUs with the highest inventory value, highlighting
-- products tying up the largest amount of working capital.
```

Equivalent SQL:

```
SELECT
    P.Product_Name,
    SUM(I.Inventory_Value) AS Inventory_Value
FROM Inventory I
JOIN Product P
    ON I.SKU_ID = P.SKU_ID
GROUP BY
    P.Product_Name
ORDER BY
    Inventory_Value DESC
LIMIT 10;
```

Expected Result:

```
-- The highest-value SKU contributes approximately ₹2.43M in inventory
-- value; several SKUs individually exceed ₹2M, indicating significant
-- working capital concentration that should be closely monitored to
-- reduce excess inventory carrying costs and optimize stock turnover.
```

END OF FILE: 02_PowerBI_SQL_Validation.sql